

Supplementary material

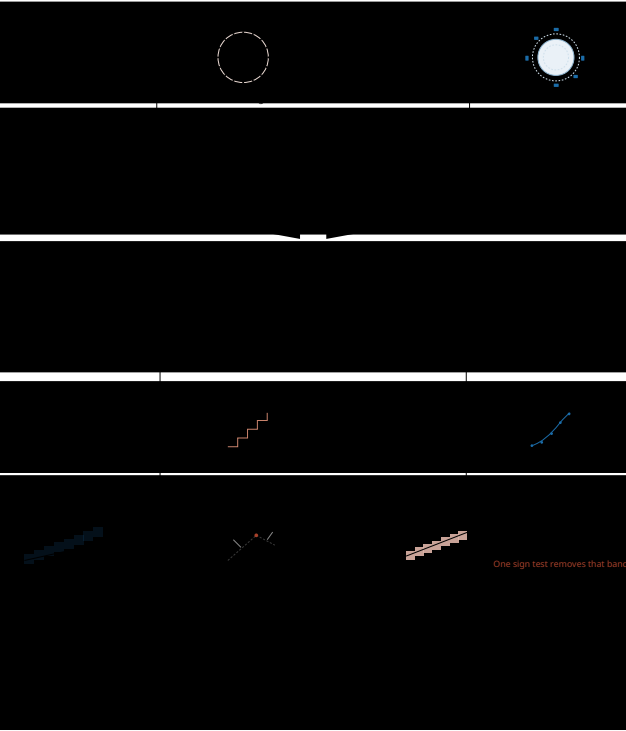
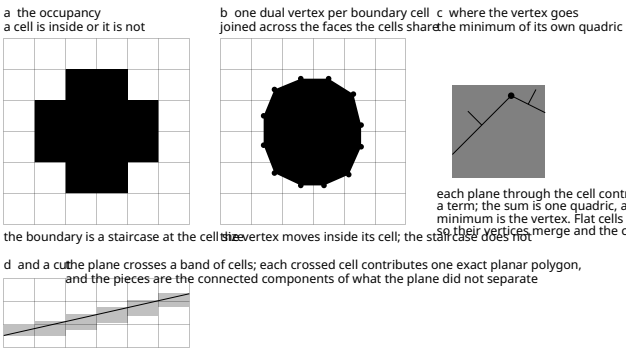
The sections the paper carries in full and the short version points to

Anonymous

The paper states every result; these are the sections whose detail did not fit a venue's length and which are kept here unchanged rather than summarised. Section numbers are the paper's own.

3.1 Preliminaries: dual grids

A dual grid is the surface counterpart of an occupancy grid. The occupancy answers inside-or-outside per cell and its boundary is a staircase; a dual grid keeps the same cells and puts one vertex inside each boundary cell, connected to the vertices of the cells it shares a face with. Connectivity is therefore fixed by the occupancy and needs no marching table; only the positions are free, and each is placed at the minimum of the quadric its own surface planes define.



4.1 What the lattice hides about its own origin

4.1.1 Four properties of the cut

Four properties the cutting code has, established on quantised lattices in the previous project before generated ones existed.

They are listed here for what they are; section 4.1.2 is the run that asks whether they survive a generated lattice, and where the two disagree numerically it is because they are different measurements of the same property rather than the same measurement twice.

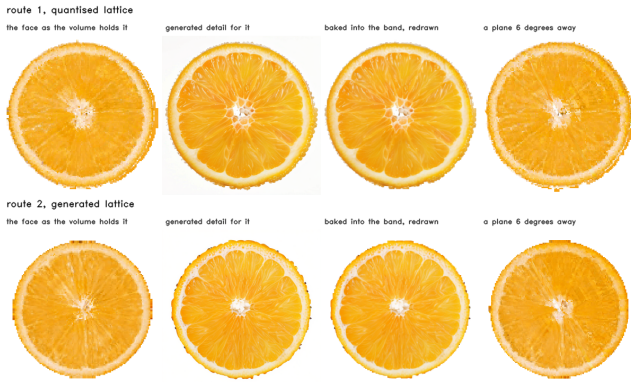
property	what it buys
Exact polyhedral fragments	0.00392 against 0.00457 relative volume error, with no subdivision against ten times the leaves
Adaptive band refinement	20.6% / 32.6% / 35.8% fewer leaves at levels 1 / 2 / 3, at equal error and identical piece counts
Several cuts at once	+77 leaves for a nearby second cut against +3,983 for an unrelated one; exact agreement with one-at-a-time
Cut-time super-resolution	95.2% of the generated detail kept at one pixel per leaf, 77.4% at six

4.1.2 Both routes

Those numbers were measured when only quantised lattices existed, so they say nothing about whether the four survive a generated one. The generated lattice differs in exactly the way that could matter here, since its cells sit on the grid these algorithms index into and the quantised one's do not. Run again on both, on the orange, with the same planes:

All four transfer. Exact fragments is marginally better on the generated lattice, where the split moves by 0.000 points rather than 0.004, because its cells sit exactly on the grid the clipping indexes into.

property	route 1, quantised	route 2, generated
Several cuts at once	+101,052 and +108,591 leaves for cuts 2 and 3; 8 pieces; all-at-once identical; folds back to 849,153 of 1,157,328	+110,152 and +113,610; 8 pieces; all-at-once identical; folds back to 894,304 of 1,215,198
Adaptive band refinement	33.5% / 32.4% / 27.4% / 16.3% fewer leaves	32.1% / 31.0% / 26.1% / 15.5%
Exact polyhedral fragments	0.004 percentage points of the object	0.000
how far the split moves		
Cut-time super-resolution	93.4% / 77.4% / 74.6%	100.0% / 76.1% / 73.2%
detail kept, at 256 / 512 / 1536 px		



Cut-time super-resolution, both routes, on the orange. Left to right: the face as the volume holds it, the detail generated for it, that detail baked into the band and the face redrawn from the cells, and a plane six degrees away drawn from the same cells. The fourth panel shows the limit of the write-back: the write-back reaches the cells the face passed through and no others, so the tilted plane meets mostly cells that never received it: it correlates more with panel 1 than with panel 3, and it has two streaks where panel 3 has thirty membranes.

Baking puts the detail in the volume rather than on the frame, which is what makes it survive the piece being turned over, but only within the band that was written. The two rows are the same procedure on a quantised and on a generated lattice.

One stage branches, and not on the route. The surface conversion colours each vertex from the nearest skin cell when skin cells are more than a fifth of the lattice, and from the nearest cell of any kind otherwise. Which side of that test a lattice falls on is a property of the object, not of how the lattice was reached: measured on the stored lattices, the skin fraction is 13.5% for the quantised orange, 68.2% for the quantised watermelon and 35.2% for the quantised doughnut, against 27.5%, 29.9% and 40.8% for the three generated ones. Five of the six are above the threshold and the quantised orange is the only one below it. An earlier version of this page said the two routes fall on opposite sides of the test; they do not. Cutting, the physics and the material field read only the three fields above.

4.3.1 Material decomposition

An object decomposes into material classes without being told what they are: cluster the learned per-cell feature, merge classes whose appearance centroids agree, and an orange separates into peel, flesh, a transition and pith. What is clustered is a learned feature and a position on a lattice, so the decomposition is indifferent to whether a cell also carries a rendering primitive. On the generated topologies it finds:

object	class	cells	mean RGB	shell fraction
orange	deep pulp	130,608 (10.6%)	0.94, 0.61, 0.03	0.00
	bright pulp	270,913 (22.0%)	0.98, 0.69, 0.10	0.00
	peel	530,696 (43.1%)	0.93, 0.62, 0.27	0.63
	transition	299,095 (24.3%)	0.96, 0.69, 0.21	0.01
watermelon	flesh	169,230 (15.9%)	0.81, 0.07, 0.09	0.00
	rind	244,766 (23.0%)	0.43, 0.52, 0.24	1.00
	lighter flesh	328,179 (30.8%)	0.92, 0.24, 0.24	0.00
	pith	323,681 (30.4%)	0.80, 0.66, 0.50	0.23
doughnut	crumb	166,915 (17.7%)	0.58, 0.42, 0.23	0.00
	lighter crumb	253,479 (26.9%)	0.71, 0.57, 0.39	0.00
	icing	365,428 (38.8%)	0.86, 0.54, 0.48	1.00
	pale crumb	157,018 (16.7%)	0.82, 0.68, 0.52	0.12



The same section three ways: as it looks, as material class, and as the stiffness that class is given. Drawn by cube lookup rather than by splatting points, so that class membership is visible as a region rather than as a scatter. The labelling shown is the weak one (shell against interior, with the interior split at its own median brightness) because it is the version a learned decomposition has to beat; the clustered classes are the table above. Red in the third panel is the stiff end of the range and blue the soft end, so the third panel is the question the next section asks: is that red band thick enough for the solver to see it?

4.3.7 Piece counts on synthetic shapes

Piece counts are checked on synthetic shapes whose answer is known, and two cases carry the test: a near-surface cut and a thin bridge. They test opposite failures. The thin cap is a separation the coarse grid cannot see, since every coarse centre falls on one side, and refinement has to find it. The one-cell bridge is the reverse: it is a connection that any labelling leaning on dilation or on a distance threshold will break, reporting two pieces for an object that is one.

case	pieces	expected
ball, plane through the centre	2	2
ball, plane clear of it	1	1
torus, plane containing the axis	2	2
torus, plane across the axis	2	2
torus, plane above the tube	1	1
dumbbell, plane through the neck	2	2
ball, cap thinner than a coarse cell	2	2 (1 without refinement)
thin bridge, plane clear of it	1	1
thin bridge, plane through it	2	2
thin bridge, plane along it	2	2

4.5.1 Cutting

The band the plane crosses is a surface through a volume, so it should scale as $N^{2/3}$. Measured at one lattice resolution it is 4.0 to 10.7% of the cells, which is consistent with that and does not establish the rate; a resolution sweep would. Where several planes meet they carve slivers whose size scales with the cell; a piece below a stated threshold rejoins the neighbour it shares the most faces with, taking the doughnut's three cuts from 18 raw pieces to 8 with the smallest kept at 2,287 cells.

object	band leaves	total leaves	pieces, one cut	pieces, three cuts
doughnut	21,760 (4.0%)	545,586	2	8 (18 raw, slivers re-joined)
orange	86,776 (9.4%)	924,252	2	8
watermelon	66,408 (10.7%)	620,162	2	n/a

4.5.3 Cross-sections

A cut face is only as good as the volume behind it, so this is the volume drawn directly: planes swept through each object, every pixel resolved by cube lookup rather than by any renderer.



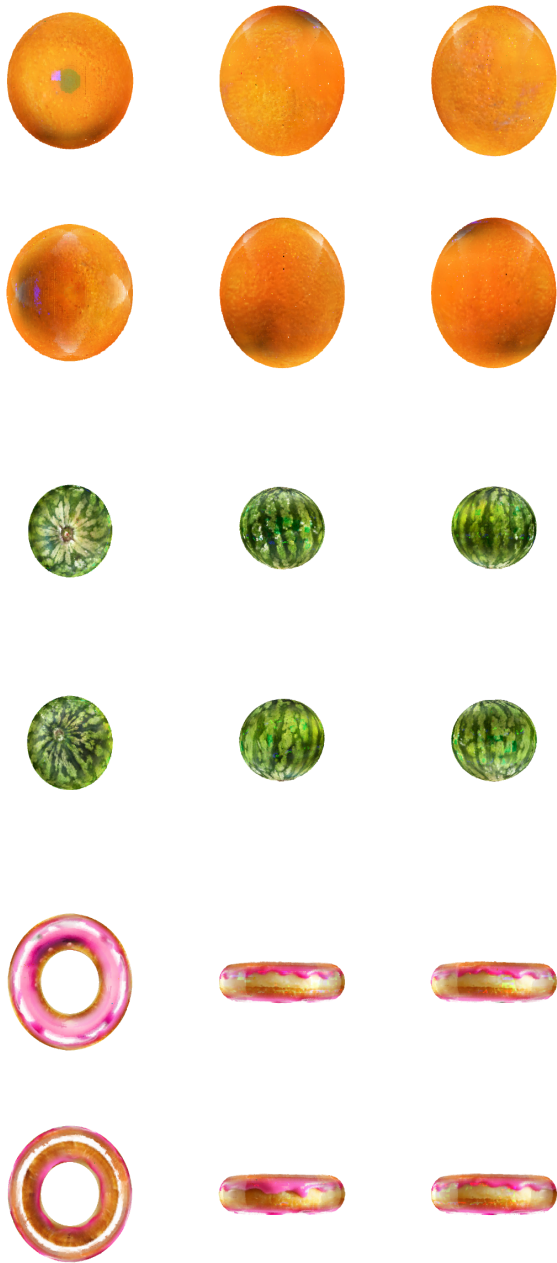
Six planes sweeping through each object, drawn by cube lookup from the filled occupancy: the smallest cell containing each pixel, decoded through the shared head. The orange's segments and pale pith, the watermelon's seeds, white pith and rind, and the doughnut's crumb are what the volume holds, not what a renderer adds. The doughnut's two lobes appear and separate as the plane moves, which is the topology and not the appearance. The table below is route 1, and it is the fill that makes a quantised occupancy solid: the orange and the watermelon go to 0.00 to 0.28% holes inside the section, and the doughnut does not, staying at 9.5 to 10.2% against 10.6 to 11.4% before. On that object the repair does essentially nothing, and the holes it reports are the ones a torus's own cross-section has where the plane misses the tube.

object	cells after filling	holes, before	holes, after
orange	877,494 → 1,059,348	14.9% / 8.7% / 6.6%	0.28% / 0.02% / 0.00%
watermelon	1,741,169 → 2,534,444	9.2% / 9.5% / 11.4%	0.08% / 0.05% / 0.10%
doughnut	797,838 → 809,318	10.6% / 10.7% / 11.4%	9.5% / 8.7% / 10.2%

The fraction of each section's own area the renderer leaves unpainted, before and after the occupancy is made solid. A cube's support is exactly its cell, so a gap is a hole in the occupancy and not a coverage parameter, which is why the first column exists at all. Those holes were always there, covered by sheer density (7,959,789 primitives, a million per unit volume) rather than by any primitive's support, which at $\sigma/h = 3 \times 10^{-5}$ reaches nothing.

4.5.4 The exterior surface

The cuts measure the interior. The claim that the appearance is independent of Gaussian training is a claim about the outside, and a shell can be right in every section and still be wrong where two views meet, because a section never looks at a seam. These are the six directions the painting used, drawn from the dual-grid surface. This is also what the exterior representation is for: a slice does not need it, because the rind a slice shows is the slab's own skin cells, but the whole object does.



Each object's exterior as an O-Voxel dual grid, from the six painting directions: top row up, front, right, bottom row down, back, left. The doughnut's icing is on its top and its crumb underneath, which is the normal being read off the field rather than off the centroid; the watermelon's stripes continue across the boundaries between views; the orange's underside is dark because the object it was rendered from was photographed sitting on a surface, and that baked lighting is carried rather than removed. Each surface vertex takes its colour from the nearest *skin* cell: the boundary is the occupancy's at any level, so 13.1% of these vertices are nearer to an interior cell than to a skin one, and colouring from the nearest cell of any kind gives that 13% whatever the interior holds.

What remains between the O-Voxel columns and the rasteriser is not capacity. The route-2 orange stores 1,231,312 primitives against the baseline's 877,494, 65.8 MiB against 46.9. What was small was the use: a cell stores fourteen numbers and the first version of this renderer read the three that are colour. Of the other eleven, seven are the scale and the rotation, which is the primitive's shape, and a shape is a reconstruction filter. On these models σ/h has median 0.26, so the rasteriser is not blurring

across cells; what it has is anisotropy, median 116 to 1, inside them.

$$h_{atc}(x) = \Sigma_{\delta \in 0,1^3} w_{\delta}(x) c_{k_0} + \delta, w_{\delta} = \Pi_a [\delta_a t_a + (1-\delta_a)(1-t_a)], t = \frac{c_{k_0} - c_{k_1}}{c_{k_0} - c_{k_1} + 1} \quad (18)$$

4.5.5 Runtime

Timings on the orange, 877,494 cells, median of three. Everything except the render rate is single-threaded NumPy on one CPU core; the render rate is one card. The collision claim holds and the cutting claim does not: five million occupancy queries a second is what a floor division and a table lookup should cost, while thirteen seconds to label the pieces is not interactive.

operation	time	rate
load the model from disk	22.8 ms	
occupancy: close and fill	1,696 ms	405,606 cells/s
one cut: refine the band, label the pieces	12,958 ms	
the cut face: exact polygons	14,246 ms	
build the collision index	151 ms	
label every particle by piece	551 ms	1,593,256 particles/s
one occupancy query over the particle set	178 ms	4,929,213 queries/s
render rate	cube volume, marching the face	Gaussian rasteriser ratio
512px	6,415.6 ms 0.16 fps	52.7 ms 18.98 fps 122×
1024px	28,871.6 ms 0.03 fps	81.7 ms 12.25 fps 353×

The third representation, the dual-grid exterior, is the one the method draws a whole object with, and it had no number here. It has one now, and it does not rescue the runtime story. There is no triangle rasteriser in this work: the surface is drawn by splatting points sampled on its triangles, three per triangle, which for the route-2 watermelon's 859,070 triangles is 5,154,420 samples.

the dual-grid exterior, route 2 watermelon	ms/frame	fps
512 px	1,057	0.95
1024 px	1,070	0.93

Single-threaded NumPy, median of three, and flat in resolution because the cost is the sample count and not the pixel count. At 512 px it is six times faster than marching the volume and twenty times slower than the Gaussian rasteriser; at 1024 px, where the march scales with pixels and this does not, it is twenty-seven times faster and thirteen times slower. A real triangle rasteriser would change this number by orders of magnitude and none is implemented, so what the table establishes is where the cost sits and not what the representation could cost.

4.2.5 Summary of differences

Everything in the table is measured here. Both prior methods release models and both were run: FruitNinja's in the frame our cameras already use, GaussianFluent's in its own frame with the camera distance matched and the upright axis chosen in its favour. FruitNinja was also reproduced from its own code with the stock checkpoint, which is the arm its published results do not cover. Its fracture and material solver are not reproduced, and GaussianFluent's own numbers for those are what its paper states.

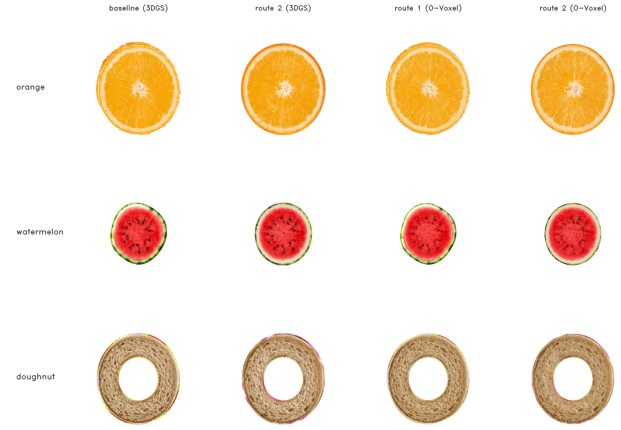
	FruitNinja	GaussianFlu-ent	this work
what holds the hidden interior	opaque atomic Gaussians	densified Gaussians, generatively guided	a cube occupancy grid
what draws a newly exposed face	the same Gaussians	the same Gaussians	a dual-grid surface, or a march into the volume
primitives, watermelon	7,959,789	1,381,100	1,741,169 route 1 / 1,065,856 route 2
stored, everything the method needs	425.1 MiB	310.8 MiB	163.5 MiB route 1 / 62.6 MiB route 2
$\rho = \sigma/h$	$3.1e-5$ (points, not blobs)	its spacing is not published	0.26 for the Gaussians a cell carries; not applicable to the cube volume, which tiles
needs a fine-tuned diffusion model	yes , and it is not released	no	no
held-out cuts through this evaluator, orange	FID 102.2 / 269.6	no orange released	74.1 to 91.5 / 213.5 to 286.8
held-out cuts through this evaluator, watermelon	FID 182.0 / 211.0	FID 170.2 / 177.5	180.9 to 183.5 / 236.7 to 262.8
the same pipeline with a stock checkpoint	FID 378.7 / 511.6 (our re-production)	needs none	FID 90.1 / 213.5
breaking into pieces	not addressed	fracture by continuum damage	connected components on face adjacency, one solver per piece
mixed materials	not addressed	the paper's subject	classes from the learned feature, stiffness by an ordering
physics	not addressed	CD-MPM, brittle fracture	MPM, piece relabelling, occupancy collision, per-piece solvers
cut geometry	not addressed	damage field	exact planar polygons, no approximation term

4.4.1 Held-out cuts

Twelve cuts per model at depths and azimuths training never sampled, scored against the photographs. Six are transverse, across the object's upright axis, and six longitudinal, through it. The O-Voxel columns register a renderer with the file that chooses the planes, so the planes, depths, cameras and order are identical by construction and only what draws the picture differs. The doughnut has one reference image per family, too few for a distribution, so it is asked instead whether every held-out transverse section still reads as an annulus.

Each score below is computed on six renders against six to twenty-three photographs. At that sample size the Inception covariance is rank-deficient and FID is dominated by its own bias [36, 37]; the photograph sets score 46.3 and 60.4 in FID against themselves split in half. On PSNR, SSIM and LPIPS the four route and renderer combinations differ by 0.29 dB, 0.010 and 0.019 on the orange and by 0.11 dB, 0.004 and 0.008 on the watermelon, and all four sit about 4.5 dB and 0.21 LPIPS below the photographs measured against each other. Differences of a

few FID points are therefore reported and not claimed; the geometric results are what is argued from.



Transverse cuts. Columns one and two are the Gaussian rasteriser on a scanned lattice and on a generated one; columns three and four draw the exposed face by marching into the cube volume until the ray enters the surviving solid, with trilinear reconstruction over the eight surrounding cell centres and no Gaussians anywhere. They do not draw a dual-grid layer: the baseline rasterises a slab about the plane, so what it makes is a slice, and a slice's rind is the slab's own skin cells, which the march already reaches.

trans-verse		route 1 (3DGS), the baseline	route 2 (3DGS)	route 1 (cube)	route 2 (cube)
orange	FID	86.4	66.5	130.5	107.9
	KID	125.0e-3	88.7e-3	208.4e-3	158.8e-3
	CLIP	31.6	31.7	30.9	32.1
watermelon	FID	187.4	180.9	188.3	167.5
	KID	233.2e-3	226.3e-3	234.9e-3	206.9e-3
	CLIP	32.3	31.9	32.5	32.2
doughnut	annulus	6/6	6/6	6/6	6/6

The doughnut has no released reconstruction to quantise, so its “route 1” is a lattice built from a generated mesh by the older shell-and-fill route, a third path kept because it is what the baseline was established on.



Longitudinal cuts, at azimuths off the trained grid. The doughnut is the case worth looking at: a plane through the axis leaves two separate lobes, and all four columns show two.

longit- udinal		route 1 (3DGS), the baseline	route 2 (3DGS)	route 1 (cube)	route 2 (cube)
orange	FID	147.6	247.3	292.6	370.2
	KID	175.3e-3	303.7e-3	437.3e-3	585.4e-3
	CLIP	31.8	34.2	31.1	33.8
water- melon	FID	266.0	244.4	314.9	263.5
	KID	331.9e-3	300.9e-3	413.6e-3	320.1e-3
	CLIP	33.4	33.7	33.6	34.7

One statistic separates the two routes where FID does not, and it is given here for what it is rather than as a result: the mean blue channel over the pulp region of a section, on the orange, which is a proxy for how grey the flesh reads. Closer to the photographs is better, and route 2 matches them transversally and overshoots longitudinally.

orange, pulp, mean blue channel	photographs	route 1	route 2
transverse	0.149	0.168	0.148
longitudinal	0.179	0.221	0.234

The cube columns are the worst in these tables and that is not a contradiction of section 4.2.2. Coverage and fidelity are different questions. Measured against the photographs' own 13.0e-3 of fine detail, the Gaussian rasteriser carries 13.8e-3 and the cube march 24.0e-3 and 30.8e-3, and the four FIDs order themselves by exactly that excess: the march resolves detail the photographs do not have, and a distribution metric punishes it for that. What section 4.2.2 establishes is that the cube volume paints the section at any resolution; what these tables establish is that at 512 px, against six photographs, it does not look more like them for it.

4.4.2 Reference-based metrics

PSNR, SSIM and LPIPS on the same cuts are in section 4.2.4, computed on a hundred planes rather than six and against the same references. They are not repeated here: at six renders the four route and renderer combinations differ by 0.29 dB, 0.010 SSIM and 0.019 LPIPS on the orange and by less than half that on the watermelon, and all four sit about 4.5 dB and 0.21 LPIPS below the photographs measured against each other. Nothing in that spread is a result.